

Juniorufgabe 2: Silbentrennung

Team-ID: 01004

Team: Wawa

Bearbeiter dieser Aufgabe:
Raphael Porsche

26. August 2025

Inhaltsverzeichnis

Programm-Ausführung.....	1
Lösungsidee.....	1
Lars Silbentrennung.....	1
Eigene Silbentrennung.....	2
Umsetzung.....	2
Lars Silbentrennung.....	2
Eigene Silbentrennung.....	3
Leistungsbewertung.....	5
Lars Silbentrennung.....	5
Eigene Silbentrennung.....	6
Werkzeuge.....	6
Beispiele.....	6
Quellcode.....	8
Lars Silbentrennung.....	8
Eigene Silbentrennung.....	9

Programm-Ausführung

Programmiersprache/Interpreter: Julia 1.12

Getestet auf: Linux, Windows

Packages: Keine

Installations und Benutzungsanweisung: Anweisung auf <https://julialang.org/install/> folgen um Julia zu installieren. Öffnen Sie ein Terminal und bewegen Sie das Arbeitsverzeichnis auf das src Verzeichnis in Junioraufgabe2. Verwenden sie diesen Befehl um das Programm auszuführen: „julia Junioraufgabe2.jl“. Die Ausgaben werden im Terminal zurückgegeben sowohl auch in ausgaben.txt im Junioraufgabe2 Verzeichnis gespeichert. Weiter Beispiele können in das „eingaben“ Verzeichnis gelegt werden.

Lösungsidee

Lars Silbentrennung

Um nach Lars Siebentrennungsregeln korrekt Silbentrennungen zu setzen, müssen wir nach jedem Buchstaben in jedem eingegebenen Wort seine 4 Regeln abfragen und danach entscheiden ob eine Trennung gesetzt wird. Die 4 Regeln können dann entweder für oder gegen eine Trennung abstimmen, oder sich enthalten. Wenn sich mehrere Regeln für oder gegen eine Trennung einsetzen, bekommt dann aber die letzte immer Recht. Hierfür gehen wir also dann durch die Eingabe durch und kontrollieren nach jedem Buchstaben ob mindestens eine Regel für eine Trennung stimmt. Wir überprüfen die Regeln in umgekehrter Reihenfolge, da spätere Regeln die vorherigen in jedem Fall überstimmen. Wenn wir bei einer Regel ankommen, die sich nicht enthält, müssen wir auch so nicht weiter durchschauen. Wir folgen das Urteil dieser einzelnen Regel und setzen eine Trennung abhängig von ihrer Entscheidung.

Eigene Silbentrennung

Nach Betrachtung von deutschen Silben, fällt einem auf, dass jede Silbe immer genau ein Vokal als eine Art von Kern besitzt. Die einzige Ausnahme sind Doppellauten wie "äu", "aa", "ee", "oo", "ei", "eu", "au", "ie", "ai", "Äu", "Ei", "Ai", "Eu", "Au", "Ie", "Aa", "Oo" und "Ee", die oft zusammen einen Kern bilden. Außerdem haben viele Silben einen weiteren Konsonanten vor dem Vokalkern. (Bei-ne, vie-le, ma-ma ...) Dies ist nicht der Fall, wenn zwischen zwei Vokalkerne keine Konsonanten vorhanden sind (Po-et, Re-ak-ti-on, Bi-o ...) und bei bestimmten Konsonanten, die zusammen dem Vokalkern vorgesetzt werden, wie z.B. 'ck', 'sch', 'ch', 'kr', 'pl', 'kn', 'tr', 'th', 'ph', 'st', 'br' und 'gl'. (A-schen-be-cher, ni-cken ...) Alle weiteren Konsonanten werden normalerweise den Silben davor als Ende dazugegeben. Diese Regeln gelten nicht bei zusammengesetzten Wörtern, die bei Wortübergänge immer Silben trennen, bei eingedeutschten Wörtern und bei bestimmten Ausnahmewörtern. Außer eine Liste von korrekter Silbentrennung von ihnen anzufertigen ist es unpraktisch diese Ausnahmefälle richtig zu trennen. Für die oben aufgezählten Regeln müssen wir also nun erst in einem Wort alle Vokalkerne finden und dann die Konsonanten verteilen, wobei wir versuchen erst jedem Vokalkern einen Konsonantenanfang zu geben, und dann die restlichen Konsonanten als Enden an davorstehende Vokalkerne anhängen. Wenn noch weitere Konsonanten vor der ersten Silbe übrigbleiben, werden sie dann auch der ersten Silbe zugeordnet.

Umsetzung

(Q1 – Q9 referenziert verschiedene Teile des Quellcodes)

Lars Silbentrennung

Für Lars Silbentrennung ist es am einfachsten 4 Funktionen zu kreieren, die das jetzige Wort und Buchstaben bekommen und dann auf die angegebenen Regeln basierend 0, 1 oder 2 zurückgeben. 2 bedeutet hier Enthaltung, bei 1 stimmt diese Regel für eine Abtrennung und bei 0 dagegen. Wir

gehen dann durch jeden Buchstaben in einem Wort durch, und fragen ab, ob wir nach dem jetzigen Buchstaben eine Trennung setzen müssen.

Lars' Regel 1:

Lars' erste Regel lautet: „Ein Wort kann zwischen zwei Konsonanten getrennt werden“. Um sie zu implementieren fragen wir ab, ob der jetzige und nächste Buchstabe ein Konsonant ist. Wenn ja, stimmt diese Regel für eine Trennung, wenn nein, dann enthält sie sich.

Lars' Regel 2:

Nach der Definition Lars' zweiter Regel: „Erster und letzter Buchstabe eines Wortes dürfen nicht abgetrennt werden“, muss diese gegen eine Trennung stimmen, wenn der jetzige Buchstabe der erste oder vorletzte ist. Ansonsten enthält sich auch diese.

Lars' Regel 3:

Lars' dritte Regel: „Bei drei aufeinanderfolgenden Konsonanten kann nur nach dem ersten Konsonanten getrennt werden“ lässt sich implementieren indem wir einfach nur sicherstellen das bei drei aufeinanderfolgenden Konsonanten nicht nach dem zweiten getrennt wird. Dafür stimmt diese Regel gegen eine Trennung, wenn der jetzige, vorherige und nächste Buchstabe alle Konsonanten sind. In jedem anderen Fall enthält sie sich.

Lars' Regel 4:

Für einer Implementation Lars' vierter Regel, müssen wir zuerst prüfen ob der jetzige Buchstabe ein Vokal ist. Daraufhin überprüfen wir ob die beiden Buchstaben danach nicht zwei Konsonanten sind. Ist beides der Fall, dann stimmt diese Regel für eine Trennsetzung, ansonsten enthält sie sich. (Q2)

```
vokale = ['a','e','i','o','u', 'ö', 'ä', 'ü', 'A', 'E', 'I', 'O', 'U', 'Ö', 'Ä', 'Ü']  
punctuation = [',','.',',','?','!']
```

Um zwischen Vokale, Konsonanten und Satzzeichen unterscheiden zu können, haben wir die zwei Vektoren ‚vokale‘ und ‚punctuation‘. Jedes Zeichen, dass in ‚vokale‘ vorkommt, ist ein Vokal. Jedes Zeichen, dass in ‚punctuation‘ vorkommt, ist ein Satzzeichen. Wenn ein Zeichen in beidem nicht vorkommt ist es in jeden Fall ein Konsonant. (Q1)

So iterieren wir dann durch jeden Buchstaben jedes Wortes und überprüfen jeweils alle Regeln. Damit bei Konflikten die letztere Regel Vorrang hat, wurden die einzelnen Regeln rückwärts in ein Vektor gelegt. Wir müssen diesen Vektor nur noch von vorne durchgehen, bis wir entweder eine 1 oder 0 zum ersten mal vorfinden. Bei 1 hat die hinterste Regel auf jeden Fall für eine Trennung gewählt. So setzen wir dann auch eine Trennung. Wird aber zuerst 0 gefunden, oder im ganzen Vektor keins von beiden, dann wird keine Trennung gesetzt. (Q3)

Dabei bauen wir langsam den Ausgang String auf, wofür jeder Buchstabe und Trennmarkieren einzeln addiert wird. Das tun wir, um nicht mit Verschiebungen arbeiten zu müssen, würden wir die Trennung im Ursprungsstring hinzufügen. Gibt es Satzzeichen, werden diese bei jedem Wort abgetrennt und zur richtigen Zeit wieder eingefügt.

Eigene Silbentrennung

Für die Umsetzung der eigenen Silbentrennung gehen wir nun erst durch jedes Wort der Eingabe einzeln durch. Für jedes Wort erstellen wir zunächst ein Vektor von Tupels, die jeweils eine Silbe repräsentieren, namens ‚kerne‘. Sie bestehen aus drei teilen, erst einem Integer, die Position des Vokalkerns im jetzigen Wort, einen Boolean, der repräsentiert ob es ein doppelter Vokalkern ist und einem weiteren Vektor von Zeichen, dass einfach die jetzige gesammelten Buchstaben der Silbe darstellt. Außerdem erstellen wir noch einen weiteren Vektor von den verschiedenen Indizes von Buchstaben, die schon auf Silben verteilt wurden, namens ‚used_indices‘. (Q4)

Dies erleichtert uns das kontinuierliche Aufbauen von den Silben, da wir nicht mit Markierungen in den eigentlichen Wörtern arbeiten müssen. Daraufhin suchen wir nach allen Kernen in dem Wort. Wie in Lars Silbentrennung, müssen wir zwischen verschiedenen Buchstaben und jetzt auch Buchstabenkombinationen unterscheiden. Dafür haben wir drei weitere Vektoren erstellt:

```
silbenkerne = ['a', 'e', 'o', 'u', 'i', 'ä', 'ö', 'ü', 'A', 'E', 'I', 'O', 'U', 'Ö', 'Ä', 'Ü']
doppelte_silbenkerne = ["äu", "aa", "ee", "oo", "ei", "eu", "au", "ie", "ai", "Äu", "Ei", "Ai", "Eu", "Au", "Ie", "Aa", "Oo", "Ee"]
klebrige_konsonanten = [['c', 'k'], ['s', 'c', 'h'], ['c', 'h'], ['k', 'r'], ['p', 'l'], ['k', 'n'], ['t', 'r'], ['t', 'h'], ['p', 'h'], ['s', 't'], ['b', 'r'], ['g', 'l']]
```

Jeder Buchstabe, der nicht in ‚silbenkerne‘ vorhanden ist, ist ein Vokal. (Satzzeichen können hier auch als Konsonanten verstanden werden, da es hier, anders als in Lars Silbentrennung, kein Problem ist wenn sie Silben zugeordnet werden.) Ansonsten werden jetzt auch noch Buchstabenkombinationen unterschieden. Um doppelte Silbenkerne zu erkennen, haben wir den Vektor ‚doppelte_silbenkerne‘ und um spezielle Konsonantenkombinationen zu erkennen, die meist oder immer zusammen an den Beginn einer Silbe gelegt werden, haben wir ‚klebrige_konsonanten‘. (Q5)

Nun gehen wir durch jeden Buchstaben des Wortes durch und suchen zuerst nach doppelten Silbenkernen, die zusammen eine Silbe bilden, markieren sie als benutzt in ‚used_indices‘ und fügen ein neues Element mit der jetzigen Position und dem „doppelte Silbenkerne“ Tag als wahr in ‚kerne‘ ein. Daraufhin führen wir das gleiche mit den restlichen einfachen Silbenkernen durch, nur mit dem „doppelte Silbenkerne“ Tag als falsch. (Q6)

Der nächste Schritt ist es, jedem Silbenkern wenn möglich ein Silbenanfang zuzuordnen. Dafür gehen wir durch jedem einzelnen Kern im ‚kerne‘ Vektor durch und überprüfen, ob der davorstehende Buchstabe als Silbenanfang addiert werden kann. Wenn dieser Teil einer speziellen Konsonantenkombination ist, werden die anderen Konsonanten auch noch zusätzlich addiert. (Q7)

Als nächsten Schritt müssen wir die restlichen Konsonanten zu Silben zuordnen. Sie sollen der unmittelbar nächsten Silbe zugeordnet werden. Um das zu vereinfachen sortieren wir den ‚kerne‘ Vektor nach deren Position im Wort, wobei die letzte Silbe im Wort nun an der ersten Stelle steht. Dies ermöglicht uns, für jeden Buchstaben, der noch keiner Silbe zugeordnet wird, durch diesen Vektor zu gehen, bis wir den ersten Silbenkern finden, der vor diesem Buchstaben im Wort steht (Q8)

Wenn wir keinen davorstehenden Silbenkern gefunden haben, bedeutet das, dass der Konsonant vor allen Silbenkernen steht, also am Anfang des Wortes ist. In diesem Fall sortieren wir ihn einfach zum ersten Silbenkern hinzu (hier: der an der letzten Stelle im sortierten ‚kerne‘ Vektor).

Zum Schluss müssen wir nun das Wort wieder mit Trennstrichen zwischen jeder Silbe zusammenbauen. Dafür gehen wir einfach durch den sortierten Kerne Vektor rückwärts durch, und addieren dem Ausgangs String jede Silbe. (Q9)

Leistungsbewertung

Lars Silbentrennung

Lars Siebentrennungsregeln funktionieren sehr schlecht. Bei den meisten Wörtern sind die Trennstriche komplett falsch gesetzt.

Eigene Silbentrennung

Obwohl das eigene Programm oft richtig die Silben trennt, kommt es dennoch regelmäßig zu Fehlern auf. Besonders bei zusammengesetzten Wörtern sind die Trennstriche häufig falsch gesetzt. Aber auch bei vielen eingedeutschten Wörtern und Sonderfällen kommt es zu falschen Trennstrichen auf. Z.b. sind manchmal mehrere Vokale eigene Silbenkerne oder bilden zusammen einen (Boot und Ko-o-pe-ra-tion). So macht das Programm ab und zu Fehler und ist nicht immer richtig.

Werkzeuge

chatgpt.com – „silben09(eigenes_Beispiel).txt“ wurde für ein längeres Beispiel mit diesem Prompt von chatgpt generiert: „Bitte erstelle einen längeren Deutschen Text. Der Inhalt ist egal.“

Beispiele

Beispiel	Ausgabe
silben01.txt	("Lars Silbentrennung: Me-i-n Na-me ist Lars und ich es-se schrec-klich ger-ne Sa-u-er-kra-u-t. ", "Eigene Silbentrennung: Mein Na-me ist Lars und ich es-se schreck-lich ger-ne Sau-er-kraut. "))
silben02.txt	("Lars Silbentrennung: De-r Ka-pi-tä-n ist eben-fal-ls Prä-si-dent de-r ör-tlic-he-n Dampf-schif-ffah-rtsgel-sel-lschaft. ", "Eigene Silbentrennung: Der Ka-pi-tän ist eben-falls

	Prä-si-dent der ört-li-chen Dampf-schiff-fahrts-ge-sell-schaft. ")
silben03.txt	("Lars Silbentrennung: Es ist ar-schkalt! ", "Eigene Silbentrennung: Es ist arsch-kalt! ")
silben04.txt	("Lars Silbentrennung: Ist da-s Au-di-o-sig-na-l i-m Ra-di-o schle-ht? ", "Eigene Silbentrennung: Ist das Au-di-o-sig-nal im Ra-di-o schlecht? ")
silben05.txt	("Lars Silbentrennung: Se-i-n Va-te-r ist Ba-u-e-r und er-nte-t Ma-i-s. ", "Eigene Silbentrennung: Sein Va-ter ist Bau-er und ern-tet Mais. ")
siblen06.txt	("Lars Silbentrennung: Ich an-gle Kar-pfe-n. ", "Eigene Silbentrennung: Ich an-gle Karp-fen. ")
silben07.txt	("Lars Silbentrennung: Wa-s sind ac-ht Hob-bits? Ei-n Hob-byte! ", "Eigene Silbentrennung: Was sind acht Hob-bits? Ein Hobby-te! ")
siblen08.txt	("Lars Silbentrennung: Fre-u-de, schö-ne-r Göt-ter-fun-ke-n, Toc-hte-r a-u-s Eli-si-u-m, Wi-r bet-re-te-n fe-u-er-trun-ke-n, Him-mlis-che, de-i-n He-i-lig-thu-m. De-i-ne Za-u-be-r bin-de-n wi-e-de-r, Wa-s di-e Mo-de streng get-he-ilt, Alle Men-sche-n wer-de-n Brü-de-r, Wo de-i-n san-fte-r Flü-ge-l we-ilt. ", "Eigene Silbentrennung: Freu-de, schö-ner Göt-ter-fun-ken, Toch-ter aus E-li-si-um, Wir be-tre-ten feu-er-trun-ken, Himm-li-sche, dein Hei-lig-thum. Dei-ne Zau-ber bin-den wie-der, Was die Mo-de streng ge-theilt, Al-le Men-schen wer-den Brü-der, Wo dein sanf-ter Flü-gel weilt. ")
silben09(eigenes_Beispiel).txt	("Lars Silbentrennung: Am frü-he-n Mor-ge-n stand e-i-ne Kat-ze a-u-f de-m Fen-ster-brett und be-o-bac-hte-te a-uf-mer-ksa-m di-e Stra-ße. Ei-n le-ic-hte-r Wind be-weg-te di-e Blät-te-r de-r al-te-n Bä-u-me, wäh-rend e-i-n Fah-rrad-fah-re-r lan-gsa-m vor-be-i-fuhr. In de-r Fer-ne hör-te ma-n da-s Klin-geln e-i-ne-r Kir-chen-gloc-ke, di-e de-n Be-ginn e-i-ne-s ne-u-e-n Ta-ge-s an-kün-dig-te. Di-e Kat-ze gäh-nte, sprang hi-nun-te-r und ver-schwand i-n de-r Küc-he, wo e-i-ne Scha-le mi-t fris-che-m Was-se-r a-u-f si-e war-te-te. Gle-ic-hze-i-tig öf-fne-te je-mand vor-sic-hti-g di-e Tü-r, tra-t hi-na-u-s und at-me-te ti-e-f di-e küh-le Luft e-i-n. Es wa-r de-r Be-ginn e-i-ne-r ru-hi-ge-n, a-be-r zug-le-ich span-nen-de-n Woc-he vol-le-r ne-u-e-r Mög-lic-hke-i-te-n. ",

	"Eigene Silbentrennung: Am frü-hen Mor-gen stand ei-ne Kat-ze auf dem Fen-ster-brett und be-o-bach-te-te auf-merk-sam die Stra-ße. Ein leich-ter Wind be-weg-te die Blät-ter der al-ten Bäu-me, wäh-rend ein Fahr-rad-fah-rer lang-sam vor-bei-fuhr. In der Fer-ne hör-te man das Klin-geln ei-ner Kir-chen-glo-cke, die den Be-ginn ei-nes neu-en Ta-ges an-kün-dig-te. Die Kat-ze gäh-n-te, sprang hi-nun-ter und versch-wand in der Kü-che, wo ei-ne Scha-le mit fri-schem Was-ser auf sie war-te-te. Gleich-zei-tig öff-ne-te je-mand vor-sich-tig die Tür, trat hi-naus und at-me-te tief die küh-le Luft ein. Es war der Be-ginn ei-ner ru-hi-gen, a-ber zu-gleich span-nen-den Wo-che vol-ler neu-er Mö-glich-kei-ten. ")
--	---

Quellcode

Lars Silbentrennung

Grundobjekte erstellen (Q1):

```
vokale = ['a','e','i','o','u', 'ö', 'ä', 'ü', 'A', 'E', 'I', 'O', 'U', 'Ö', 'Ä', 'Ü']
punctuation = ['.',',',' ','?','!']
```

Umsetzung der 4 Regeln (Q2):

```
function lr_1(word, pos::Int)
    if !(word[pos] in vokale) && !(word[pos+1] in vokale)
        return 1
    else
        return 2
    end
end

function lr_2(word, pos::Int)
    if pos == 1 return 0 end
    if pos == (length(word)-1) return 0 end
    return 2
end

function lr_3(word, pos::Int)
    if pos < 2 return 2 end
    if !(word[pos - 1] in vokale) && !(word[pos] in vokale) && !(word[pos + 1] in vokale)
        return 0
    else
        return 2
    end
end
```

```

    end
end

function lr_4(word, pos::Int)
    if pos + 2 > length(word) && (word[pos] in vokale) return 1 end
    if word[pos] in vokale && (word[pos+1] in vokale || word[pos+2] in vokale)
        return 1
    else
        return 2
    end
end
end

```

Auswerten der Entscheidungen der Regeln (Q3):

```

rule_decisions = [lr_4(word,idx), lr_3(word,idx), lr_2(word,idx),
lr_1(word,idx)]
for decision in rule_decisions
    if decision == 1
        output *= ' - '
        break
    elseif decision == 0
        break
    end
end
end

```

Eigene Silbentrennung**Grundobjekte erstellen (Q4):**

```

kerne = Vector{Tuple{Int, Bool, Vector{Char}}{}}()
used_indices = Vector{Int}{}

```

Zuordnung von Silbenkernen, doppelten Silbenkernen und klebrigen Konsonanten (Q5):

```

silbenkerne = ['a', 'e', 'o', 'u', 'i', 'ä', 'ö', 'ü', 'A', 'E', 'I', 'O', 'U', 'Ö', 'Ä', 'Ü']
doppelte_silbenkerne = ["äu", "aa", "ee", "oo", "ei", "eu", "au", "ie", "ai", "Äu", "Ei", "Ai", "Eu",
"Au", "Ie", "Aa", "Oo", "Ee"]
klebrige_konsonanten = [['c', 'k'], ['s', 'c', 'h'], ['c', 'h'], ['k', 'r'], ['p', 'l'], ['k', 'n'], ['t', 'r'], ['t', 'h'], ['p', 'h'],
['s', 't'], ['b', 'r'], ['g', 'l']]

```

Silbenkerne finden (Q6):

```

for idx in 1:(length(word)-1)
    if (word[idx] * word[idx+1]) in doppelte_silbenkerne
        push!(kerne, (idx, true, [word[idx]]))
        push!(used_indices, idx)
    end
end
end
for (idx, char) in enumerate(word)
    if char in silbenkerne && !is_doppel_kerne(kerne, idx)
        push!(kerne, (idx, false, [word[idx]]))
    end
end

```

```

        push!(used_indices, idx)
    end
end

```

Anfänge an Silbenkerne anhängen (Q7):

```

for kern in kerne
    if kern[1] < 2 continue end
    if (kern[1] - 1) in get_idx_kerne(kerne) continue end
    if is_doppel_kerne(kerne, kern[1] - 1) continue end

    stop = false
    for kon in klebrige_konsonanten
        if kern[1] <= length(kon) continue end
        if word[(kern[1]-length(kon)):(kern[1]-1)] == kon
            prepend!(kern[3], kon)
            append!(used_indices, (kern[1]-length(kon)):(kern[1]-1))
            stop = true
            break
        end
    end
    stop && continue
    pushfirst!(kern[3], word[kern[1] - 1])
    push!(used_indices, kern[1] - 1)
end

```

Übrigbleibende Konsonanten anhängen (Q8):

```

sort!(kerne, lt=(x,y) -> isless(x[1], y[1]), rev=true)
add_to_beginning = Vector{Char}()
for idx in eachindex(word)
    if idx in used_indices continue end
    i = findfirst(x -> x[1] < idx, kerne)
    if isnothing(i)
        push!(add_to_beginning, word[idx])
    else
        push!(kerne[i][3], word[idx])
    end
end
end
prepend!(kerne[end][3], add_to_beginning)

```

Wort wieder zusammenbauen (Q9):

```

for (idx, kern) in enumerate(reverse(kerne))
    output *= String(kern[3])
    if idx != length(kerne)
        output *= '-'
    end
end

```

```
end  
output *= ''
```